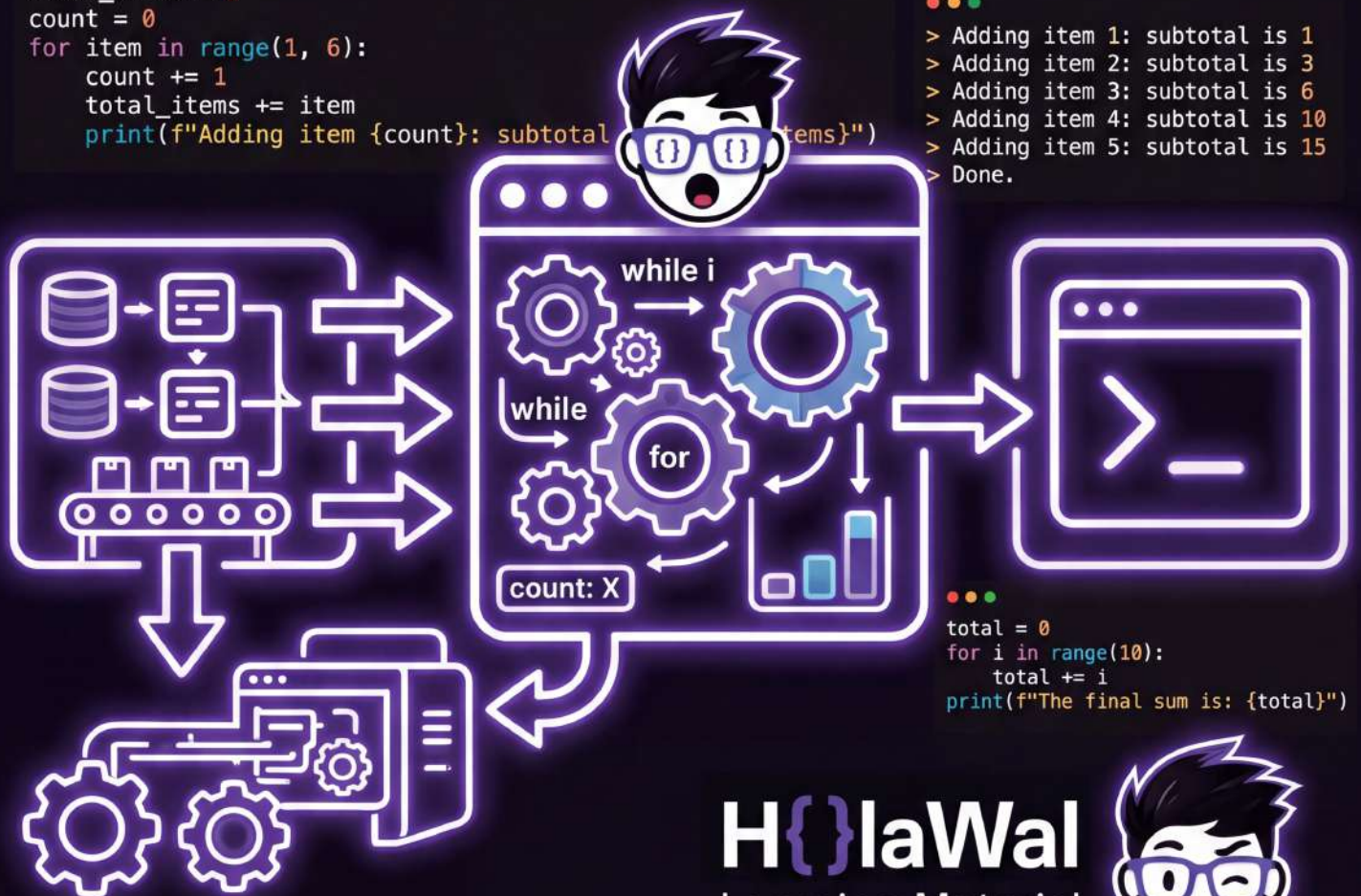


MÓDULO 3: INTRODUCCIÓN A PYTHON

El Poder de la Automatización — Bucles (while y for), Contadores y Acumuladores

```
total_items = 0
count = 0
for item in range(1, 6):
    count += 1
    total_items += item
    print(f"Adding item {count}: subtotal {total_items}")
```

```
> Adding item 1: subtotal is 1
> Adding item 2: subtotal is 3
> Adding item 3: subtotal is 6
> Adding item 4: subtotal is 10
> Adding item 5: subtotal is 15
> Done.
```



■ CURSO PYTHON DESDE CERO — DÍA 3

Módulo 3: El Poder de la Automatización — Bucles (while y for), Contadores y Acumuladores

Guía Integral Teórica, Metodológica y Práctica

■ **Propósito del Módulo:** Esta guía combina la **teoría conceptual profunda** (explicando la mecánica interna, la gestión de memoria y el flujo de ejecución de los bucles) con la **metodología de ingeniería de software basada en problemas** (Contexto Real, Requerimientos Técnicos, Implementación Comentada y Casos de Prueba).

1. Fundamentos Teóricos: ¿Por qué Existen los Bucles?

En programación, las instrucciones secuenciales se ejecutan una sola vez de arriba hacia abajo. Sin embargo, en el mundo real, casi todos los problemas computacionales requieren procesar colecciones de datos, esperar eventos del usuario o repetir cálculos. Un **bucle** (o estructura de control iterativa) es una construcción sintáctica que permite re-ejecutar un bloque de código múltiples veces de forma controlada sin duplicar líneas de código fuente (cumpliendo el principio *DRY: Don't Repeat Yourself*).

Conceptos Clave de la Mecánica de un Bucle:

- **Iteración:** Cada una de las vueltas o ejecuciones individuales del bloque interno del bucle.
- **Condición de Parada:** Expresión booleana que determina si el bucle continúa ejecutándose (True) o finaliza (False).
- **Cuerpo del Bucle:** Conjunto de instrucciones indentadas (4 espacios) que se ejecutan en cada iteración.

2. El Bucle 'while': Repetición Basada en Condición Lógica

El bucle while ('mientras que') evalúa una condición antes de entrar al cuerpo de instrucciones. Si la condición es verdadera, ejecuta el bloque y vuelve a evaluar la condición. El ciclo se repite indefinidamente hasta que la condición se convierta en falsa.

Sintaxis Estándar del bucle while:

```
while <condicion_booleana>:  
    # Instrucciones a repetir (Cuerpo del bucle)  
    # Modificación de la variable de control (¡Indispensable!)
```

■ **El Peligro del Bucle Infinito:** Si el programador olvida modificar la variable que interviene en la condición, la condición siempre será True, haciendo que el programa nunca termine y consuma el 100% de la CPU.

Caso de Estudio 1: Monitoreo Térmico con Bucle while

■ **Contexto del Problema:** Una clínica médica almacena vacunas que deben conservarse a temperaturas no negativas. Se requiere un programa que registre continuamente lecturas de temperatura en grados Celsius (°C) hasta que el sensor detecte un valor bajo cero (< 0 °C), momento en el cual debe detener el monitoreo y activar una alerta de congelamiento.

■ **Requerimientos:** Entrada: lecturas de temperatura sucesivas (float). Proceso: bucle while temp >= 0. Salida: confirmación de lectura normal o alarma de congelamiento.

```
# Implementación: Monitoreo Térmico con while  
print("=== SISTEMA DE MONITOREO DE REFRIGERACIÓN MÉDICA ===\n")  
temp = float(input("Ingrese temperatura inicial (°C): "))  
  
while temp >= 0:  
    print(f"✓ Temperatura segura registrada: {temp:.1f} °C")  
    temp = float(input("Ingrese siguiente temperatura (°C): "))  
  
print(f"\n■ ¡ALERTA CRÍTICA! Temperatura bajo cero: {temp:.1f} °C (Monitoreo detenido).")
```

3. El Bucle 'for' y la Función range(): Iteración Determinada

A diferencia de otros lenguajes donde el bucle for utiliza un contador manual, en Python el bucle for está diseñado para **iterar directamente sobre secuencias**. La función integrada range() genera secuencias aritméticas inmutables.

Anatomía de la Función range(start, stop, step):

- range(stop): Comienza en 0 y llega hasta stop - 1. Ej: range(5) → 0, 1, 2, 3, 4.
- range(start, stop): Comienza en start y termina en stop - 1. Ej: range(1, 6) → 1, 2, 3, 4, 5.
- range(start, stop, step): Avanza de step en step. Ej: range(2, 11, 2) → 2, 4, 6, 8, 10.
- **Paso Negativo (Cuenta Regresiva)**: Ej: range(5, 0, -1) → 5, 4, 3, 2, 1.

Caso de Estudio 2: Generador de Tablas Comerciales con Bucle for

■ **Contexto del Problema:** Un distribuidor mayorista necesita imprimir en consola la lista de cotizaciones escalonada por volumen (de 1 a 12 unidades) para cualquier producto a partir de su precio unitario base.

■ **Requerimientos:** Entrada: producto (str) y precio (float). Proceso: for cant in range(1, 13): calculando cant * precio. Salida: tabla estructurada alineada.

```
# Implementación: Cotizaciones por Volumen con for
producto = input("Nombre del producto: ")
precio_base = float(input("Precio unitario base ($): "))

print(f"\n{'Cant.':<6}{'Precio Unit.':<16}{'Total a Pagar':<12}")
print("-" * 36)
for cant in range(1, 13):
    total = cant * precio_base
    print(f"{cant:<6}x ${precio_base:<13.2f}= ${total:<10.2f}")
```

4. Algoritmos Fundamentales: Contadores, Acumuladores y Control de Flujo

- **Contador (c += 1):** Variable entera inicializada generalmente en 0 que se incrementa en una cantidad fija (usualmente +1) en cada iteración para llevar el registro de cuántas veces ocurrió un suceso.
- **Acumulador (suma += valor):** Variable numérica inicializada en 0 (o 1 para multiplicaciones) que suma valores variables en cada iteración.
- **Sentencia break:** Fuerza la salida inmediata y definitiva del bucle en ejecución.
- **Sentencia continue:** Aborta la iteración actual y salta directamente a la siguiente evaluación de condición.

Caso de Estudio 3: Cierre de Caja Diario con Contador y Acumulador

■ **Contexto del Problema:** Un comercio registra tickets de venta uno tras otro hasta ingresar el valor centinela 0.

■ **Requerimientos:** Ignorar montos negativos con continue, cerrar el bucle con break al recibir 0 y mostrar el total facturado, número de tickets y ticket promedio.

```
# Implementación: Cierre de Caja con Control de Flujo
total_dinero = 0.0
total_tickets = 0

while True:
    monto = float(input("Monto ticket ($) [0 para terminar]: "))
    if monto == 0:
        break
    if monto < 0:
        print("■ Monto no válido.")
        continue
    total_dinero += monto
    total_tickets += 1

promedio = total_dinero / total_tickets if total_tickets > 0 else 0.0
print(f"\nFacturación ({total_tickets} tickets): ${total_dinero:.2f} | Promedio: ${promedio:.2f}")
```

5. Proyecto Guiado: Juego del Número Secreto con Límite de Intentos

■ **Contexto y Requerimientos:** Generar un número aleatorio con `random.randint(1, 100)`. El jugador dispone de un máximo de 7 intentos. En cada intento fallido, el programa debe indicar si el número secreto es Mayor o Menor. El bucle se controla con `while intentos < 7 and not acertado:`.

```
# Implementación Completa: Videojuego del Número Secreto
import random

secreto = random.randint(1, 100)
max_intentos = 7
intentos = 0
acertado = False

print(f"=== ■ ADIVINA EL NÚMERO (1-100) – {max_intentos} INTENTOS ===\n")
while intentos < max_intentos and not acertado:
    intentos += 1
    intento = int(input(f"Intento #{intentos}/{max_intentos} >> Ingresa número: "))
    if intento == secreto:
        acertado = True
    elif intento < secreto:
        print("■■■ ¡Pista: El número secreto es MAYOR!")
    else:
        print("■■■ ¡Pista: El número secreto es MENOR!")

if acertado:
    print(f"\n■ ¡VICTORIA! Acertaste el número {secreto} en {intentos} intentos.")
else:
    print(f"\n■ DERROTA: Se agotaron los intentos. El número era {secreto}.")
```

6. Banco de Ejercicios Propuestos con Requerimientos

- **Ejercicio 3.1 — Filtro de Números Pares:** Imprime todos los números pares del 1 al 50 usando `range(2, 51, 2)`.
- **Ejercicio 3.2 — Sumatoria Acumulada (1 a N):** Pide un entero positivo `n` y calcula la suma acumulada de 1 hasta `n`.
- **Ejercicio 3.3 — Auditoría de Notas:** Pide `N` notas y cuenta cuántos alumnos aprobaron (≥ 11) y cuántos desaprobaron usando contadores.
- **Ejercicio 3.4 — PIN de Cajero con 3 Intentos:** Valida el PIN 2026. Si falla 3 veces consecutivas, bloquea el sistema.
- **Ejercicio 3.5 — Calculadora de Factorial:** Calcula $n! = 1 \times 2 \times \dots \times n$ con un acumulador multiplicativo.

■ DESAFÍO DÍA 3: Simulador de Cajero Automático con Menú Persistente

Crea un programa con saldo inicial de \$1,000 USD y un menú infinito (`while True`) con opciones: (1) Consultar Saldo, (2) Depositar (validando positivo), (3) Retirar (validando fondos) y (4) Salir con `break`.